

BUGMXT SI

Bug Triage Report

5-layer code integrity scan · ATANDA Command Centre · Account-management commit ffc9d3e

BUGMXT SI — Bug Triage Report

Target: ATANDA Command Centre — commit ffc9d3e (Account management + auth-safety hardening)

Engine: BUGMXT SI v1.0 (JCSE 46 / Platinum)

Model: claude-sonnet-4-6 (4-phase scan)

Scan duration: 145.2s total

Tokens: 67161 in / 6298 out

Generated: 2026-05-19T01:41:16.698Z

Executive Summary

The change set is structurally sound and the core anti-resurrection logic is correct, but three issues escaped architect review. The most serious is a post-Clerk-delete window in `POST /me/delete` where a Drizzle hard-delete failure leaves a locally-deleted-Clerk / still-local-DB split that has no recovery path and no error response to the client. A secondary logic flaw in the React form hydration guard means the `displayName` field will silently fail to pre-populate for users whose `displayName` is an empty string (edge case, but real). One OpenAPI schema gap — `DeleteAccountResult.deletedAt` is marked optional in the spec but the implementation always emits it — creates a client-contract mismatch that Orval will propagate into generated types. Security posture is meaningfully improved over the prior state; no critical blockers to shipping, but two issues should be patched before the next release cut.

Layer 1 — Syntax Sweep

- **[SYNTAX-001]** | Severity: LOW | File: `artifacts/api-server/src/routes/me.ts` | Lines: 279–281

The expression `void commandCentreSubscribersTable;` is a no-op statement used as a workaround to suppress a TypeScript/ESLint unused-import warning. `commandCentreSubscribersTable` is imported but never referenced in any Drizzle query within this file — the `req.subscriber!` object used throughout is populated by `requireAuth/ensureSubscriber`, not by a query here. The suppression comment ("used implicitly by ... re-export") is factually incorrect; there is no re-export in this file. The real fix is to remove the import rather than silence the diagnostic. As written, this compiles cleanly but the dead import and void statement will confuse static analysis and any AI agent parsing this module.

Recommendation: Remove `commandCentreSubscribersTable` from the import list on line 10 and delete the `void commandCentreSubscribersTable;` statement.

- **[SYNTAX-002]** | Severity: LOW | File: `lib/api-spec/openapi.yaml` | `components/schemas/DeleteAccountResult`

The `deletedAt` property is defined in the schema but **not** listed in the `required` array (only `ok` is required). The implementation at `POST /me/delete` unconditionally emits `deletedAt: new Date().toISOString()`, making `deletedAt` always present. The omission from `required` is a schema authoring error — Orval will generate `deletedAt?: string` (optional) in the TypeScript client types, diverging

from runtime reality. This is a contract inconsistency detectable at the syntax/schema level.

Recommendation: Add `deletedAt` to the required array of `DeleteAccountResult`:

```
```yaml
required: [ok, deletedAt]
```
```

No further syntax-layer findings.

Layer 2 — Logic & Outcome Audit

[LOGIC-001] `ClerkIdentityNotFoundError` defined after first use in call-site error handler

The class `ClerkIdentityNotFoundError` is declared at line ~64 of `auth.ts` (after the `requireAuth` function body that catches it at line ~42). In JavaScript/TypeScript, class declarations are **not** hoisted like function declarations — the `catch (err) { if (err instanceof ClerkIdentityNotFoundError)` check inside `requireAuth` references the class before it is defined in source order, which will throw a `ReferenceError` at runtime on first auth attempt.

- **File:** `artifacts/api-server/src/lib/auth.ts`
- **Location:** `requireAuth` (~line 42) catches `ClerkIdentityNotFoundError`; class is declared at ~line 64
- **Expected:** `instanceof` check resolves correctly; a deleted-user token gets a 401.
- **Actual:** `ReferenceError: ClerkIdentityNotFoundError is not defined` thrown at the `instanceof` check, causing a 503 (or an unhandled crash) for every auth request until the code is fixed.
- **Fix pathway: Patch** — Move the class `ClerkIdentityNotFoundError` declaration to the top of the file (or at minimum before `requireAuth`).

[LOGIC-002] Race window between Clerk delete and local hard-delete allows JIT shell-recreation

After `clerkClient.users.deleteUser(clerkUserId)` succeeds (step 2) but before `db.delete(usersTable)` completes (step 3), any concurrent in-flight authenticated request from the same session will call `requireAuth !ensureLocalUser`. Because the local row still exists at that instant, `ensureLocalUser` returns it and the request proceeds normally. More critically: if the local `db.delete` then fails mid-flight (transient DB error), the Clerk identity is already gone but the local user row survives, leaving a permanently orphaned local record that can never authenticate again and cannot be self-deleted by the user (their Clerk identity is gone).

- **File:** `artifacts/api-server/src/routes/me.ts`, `router.post("/me/delete", ...)` steps 2–3
 - **Expected:** Atomic Clerk + local deletion; no window for concurrent access or partial-delete orphan.
 - **Actual:** Non-atomic two-phase delete with a race window; a DB failure after Clerk delete permanently orphans the local row with no recovery path.
 - **Fix pathway: Refactor** — Mark the local user as `deletionPending = true` (soft-delete flag) before touching Clerk, so concurrent requests are rejected immediately. Wrap the local `db.delete` in a `try/catch`; if it fails post-Clerk-delete, log a CRITICAL alert for manual reconciliation (the user cannot self-heal). Alternatively, invert order: soft-delete local row first, then delete from Clerk, then hard-delete local row — keeping Clerk delete as the point of no return.
-

[LOGIC-003] Stripe subscription cancel not idempotent across retries — already-cancelled state not checked before calling cancel

The code calls `stripe.subscriptions.cancel(s.stripeSubscriptionId)` and only treats the response as benign if `statusCode === 404` or `code === "resource_missing"`. However, Stripe also returns a 400 with code: `"subscription_already_canceled"` for subscriptions that are in canceled state. This is not caught as benign, so a user with an already-cancelled subscription who attempts account deletion will receive a spurious 502 error and be blocked from deleting their account.

- **File:** `artifacts/api-server/src/routes/me.ts`, Stripe cancel block (~line 155)
 - **Expected:** Any already-terminal subscription state (cancelled, expired) is treated as a no-op and deletion proceeds.
 - **Actual:** A `subscription_already_canceled` Stripe error propagates as a 502, blocking account deletion.
 - **Fix pathway: Patch** — Add `code === "subscription_already_canceled"` to the benign condition, or alternatively check `s.status` (from the local subscriber record) before calling Stripe and skip the cancel call if status is already canceled or `incomplete_expired`.
-

[LOGIC-004] ``onConflictDoNothing`` in ``ensureLocalUser`` can return ``undefined`` ``created``, and the fallback re-read is not guarded against the strict-mode error path

When the `INSERT ... ON CONFLICT DO NOTHING` fires (concurrent JIT creation), `created` is `undefined`. The code then falls through to a re-read query. However, this re-read is only reached if the Clerk `getUser()` call **succeeded** — which is now mandatory in strict mode. If Clerk `getUser()` throws a non-404 error, the function throws immediately and never reaches the insert. That is correct. But there is a subtler problem: if Clerk returns the user successfully, the insert conflicts (another thread won the race), `created` is `undefined`, the re-read executes — and if the re-read also returns 0 rows (e.g., the winning concurrent insert was itself inside a transaction that rolled back), the function implicitly returns `undefined` (falling off the end), not a `User`. TypeScript's return type annotation `Promise<User>` masks this; at runtime `req.localUser` would be `undefined`, causing downstream null-dereference crashes.

- **File:** `artifacts/api-server/src/lib/auth.ts`, `ensureLocalUser`, post-insert re-read (~line 103+)
 - **Expected:** Function always returns a valid `User` or throws.
 - **Actual:** In a narrow concurrent-insert-rollback scenario, function returns `undefined` silently, crashing downstream code that dereferences `req.localUser`.
 - **Fix pathway: Patch** — Add an explicit guard after the re-read: `if (!reread[0]) throw new Error("ensureLocalUser: local user vanished after conflict — this should never happen");` so the failure is loud and traceable rather than a silent `undefined` propagation.
-

[LOGIC-005] ``PATCH /me/profile`` does not guard against updating a user who has been soft-deleted or is mid-deletion

Because there is no soft-delete flag, a user who has initiated deletion (Clerk gone, local row not yet deleted) can still hit `PATCH /me/profile` with a valid-but-stale JWT. `requireAuth` will succeed (local row still exists, Clerk verification may still pass transiently depending on JWT TTL), and `db.update(usersTable).set({ displayName }).where(eq(usersTable.id, u.id))` will update a row that is about to be hard-deleted. This is a minor logical inconsistency but more importantly it re-entrenches the race described in [LOGIC-002].

- **File:** `artifacts/api-server/src/routes/me.ts`, `router.patch("/me/profile", ...)`
- **Expected:** No mutations possible on an account mid-deletion.

- **Actual:** Profile can be mutated concurrently with deletion; last writer wins on a row scheduled for deletion.
- **Fix pathway: Patch** — Resolved entirely by the soft-delete flag recommended in [LOGIC-002]; once that flag is in place, `requireAuth` (or each mutating route) should reject requests where `user.deletionPending === true`.

[LOGIC-006]

Layer 3 — HARP (Human + AI Readability)

- **[HARP-AI-001]** | Severity: MEDIUM | File: `auth.ts`

`ClerkIdentityNotFoundError` is declared **after** the `requireAuth` function that catches it. Any agent reading the file top-to-bottom encounters a catch clause for a class that does not yet exist at that point in the source. TypeScript resolves this via hoisting of `class` declarations in the same scope only when they share a block — here they do not share an immediately-obvious scope sequence, and the ordering will confuse both human reviewers doing a sequential read and any AI agent performing static analysis without full AST awareness. **Recommendation:** Hoist `ClerkIdentityNotFoundError` to the top of the module, before `requireAuth`.

- **[HARP-AI-002]** | Severity: MEDIUM | File: `auth.ts`

The dual-field status probe `(err as { status?: number; statusCode?: number }).status ?? (err as { status?: number; statusCode?: number }).statusCode` is duplicated verbatim in two separate files (`auth.ts` and `me.ts`). There is no shared helper (e.g. `extractHttpStatus(err: unknown): number | undefined`). An AI agent encountering this pattern in a third file has no canonical reference to follow and will reproduce the inline cast again, perpetuating an untyped, unchecked pattern.

Recommendation: Extract to `src/lib/http-status-from-error.ts` with a named export.

- **[HARP-AI-003]** | Severity: LOW | File: `auth.ts`

The fire-and-forget welcome-email block uses `void (async () => { ... })()`. This idiom is uncommon enough that agents unfamiliar with the project's style will flag it as a potential unhandled-promise leak or misread the `void` as a type annotation. The inner swallowed `catch {}` has no log call, so a persistent email-service failure produces zero observability signal. **Recommendation:** Name the helper (`sendWelcomeEmailBestEffort(email, displayName)`) and add at minimum a `req.log?.warn` (or a module-level logger call) inside the catch.

- **[HARP-AI-004]** | Severity: MEDIUM | File: `me.ts` (bottom of file)

```
```ts
```

```
// Suppress unused import warning — used implicitly by commandCentreSubscribersTable re-export.
```

```
void commandCentreSubscribersTable;
```

```
```
```

This comment is factually misleading: `commandCentreSubscribersTable` **is** used explicitly in the `DELETE /me/delete` route (`eq(commandCentreSubscribersTable...)` is never called directly — the table is accessed only through the cascade, so the *comment* implies a re-export that does not exist in this diff). An AI agent reading this will infer a re-export contract that it cannot verify, leading to hallucinated dependency assumptions. If the import is genuinely unused, remove it; if it is needed for a side-effect registration, document that side-effect explicitly. **Recommendation:** Remove the dead `void` statement and the import if it is unused, or document the actual side-effect clearly.

- **[HARP-AI-005]** | Severity: LOW | File: `me.ts` — PATCH `/me/profile`

The variable `s` (subscriber) is extracted from `req.subscriber!` but is **never used** inside the profile-update handler body — it does not appear in the Drizzle update call. However, the *response* body reads `s.tier`,

`s.status`, etc., so it is used in the response shape. The single-character alias `S` at the top of a 40-line handler, immediately adjacent to `u`, makes the data-flow non-obvious. **Recommendation:** Rename to `subscriber / user` to match the domain vocabulary used everywhere else in the diff.

- **[HARP-HUMAN-001]** | Severity: LOW | File: `account.tsx`

The form-hydration block is placed **outside any `useEffect``** and runs on every render inside the component body:

```
``tsx
if (me && displayName === "" && !updateProfile.isPending) {
  setDisplayName(me.displayName ?? "");
}
``
```

This is a React anti-pattern (state mutation during render). While React tolerates it in narrow cases (the condition usually becomes false after the first state update), it is confusing to any human reviewer or AI agent trained on canonical React idioms and may cause subtle double-render issues. **Recommendation:** Move into a `useEffect(() => { ... }, [me])` with an early-return guard.

- **[HARP-HUMAN-002]** | Severity: LOW | File: `account.tsx` — Danger Zone card

The `CardDescription` reads: *"Any active Stripe subscription should be cancelled first on the Billing page."* However, the backend (`DELETE /me/delete`) **already cancels the subscription automatically** (step 1 of the deletion flow). The UI copy actively contradicts the server behaviour, creating user confusion and potential support burden. **Recommendation:** Update copy to: *"Any active subscription will be cancelled automatically when your account is deleted."*

- **[HARP-AI-006]** | Severity: LOW | File: `openapi.yaml` — `MyDataExport` schema

All nested objects (`user`, `subscriber`, `sessions[*]`, `artifacts[*]`, `badges[*]`) are typed as `additionalProperties: true` with no defined property list. An Orval code-generation pass over this schema will produce `Record<string, unknown>` for every field, giving zero type guidance to the generated client and stripping all autocompletion from consuming code. **Recommendation:** Inline or `$ref` the actual property shapes (or at minimum list required fields) so generated types are useful.

- **[HARP-HUMAN-003]** | Severity: LOW | File: `auth.ts` — `ensureLocalUser`

The function has no JSDoc block. It is a cross-cutting authentication primitive called from `requireAuth` (a public middleware), yet its contract — "throws `ClerkIdentityNotFoundError` on 404, throws generic `Error` on any other Clerk failure, returns existing or newly-created `User`" — is entirely implicit. A new engineer or agent onboarding to this codebase must read the full implementation to understand the throw contract.

Recommendation: Add a JSDoc block describing parameters, return type, and the two throw cases.

- **[HARP-AI-007]** | Severity: MEDIUM | File: `me.ts` — `DELETE /me/delete`

The user-facing error message on Stripe unavailability reads:

> *"Billing service is temporarily unavailable; we cannot cancel your subscription right now. Please retry shortly."*

The user-facing message on Clerk failure reads:

> *"Identity provider could not delete your account. Try again."*

Neither message gives the user a path to escalation (support link, status page). More critically for AI parseability: the 502 status is used for both a transient-availability failure **and** a non-transient Stripe API error (non-404, non-`resource_missing`). A monitoring agent consuming error logs cannot distinguish "Stripe is down" from "the subscription ID is in a terminal state Stripe rejects" using status code alone. **Recommendation:** Use 503 for transient unavailability and 502 (or 422

Layer 5 — EAL Bayesian Triage

EAL Score = (Severity × 0.35) + (Likelihood × 0.30) + (Blast Radius × 0.20) + (Detectability × 0.15). All scores on a 1–5 scale; higher = worse.

| ID | Layer | Severity (1-5) | Likelihood (1-5) | Blast Radius (1-5) | Detectability (1-5) | EAL Score | Priority |
|------------------|-------|----------------|------------------|--------------------|---------------------|-----------|----------|
| [LOGIC-001] | L2 | 5 | 5 | 5 | 3 | **4.70** | CRITICAL |
| [LOGIC-002] | L2 | 5 | 3 | 5 | 4 | **4.30** | CRITICAL |
| [LOGIC-004] | L2 | 4 | 3 | 4 | 5 | **3.85** | HIGH |
| [LOGIC-003] | L2 | 3 | 4 | 3 | 3 | **3.30** | HIGH |
| [HARP-AI-001] | L3 | 4 | 5 | 3 | 2 | **3.65** | HIGH |
| [SYNTAX-002] | L1 | 3 | 5 | 3 | 2 | **3.25** | HIGH |
| [HARP-AI-002] | L3 | 2 | 4 | 3 | 3 | **2.80** | MEDIUM |
| [HARP-AI-004] | L3 | 2 | 5 | 2 | 2 | **2.65** | MEDIUM |
| [HARP-AI-007] | L3 | 3 | 3 | 2 | 3 | **2.80** | MEDIUM |
| [LOGIC-005] | L2 | 2 | 3 | 3 | 4 | **2.75** | MEDIUM |
| [HARP-AI-006] | L3 | 2 | 5 | 2 | 1 | **2.60** | MEDIUM |
| [HARP-HUMAN-002] | L3 | 2 | 4 | 2 | 2 | **2.40** | MEDIUM |
| [HARP-HUMAN-001] | L3 | 2 | 3 | 1 | 2 | **2.05** | LOW |
| [HARP-AI-003] | L3 | 2 | 2 | 1 | 3 | **1.95** | LOW |
| [HARP-AI-005] | L3 | 1 | 5 | 1 | 2 | **1.90** | LOW |
| [HARP-HUMAN-003] | L3 | 1 | 3 | 2 | 3 | **1.85** | LOW |
| [SYNTAX-001] | L1 | 1 | 5 | 1 | 1 | **1.80** | LOW |

> **Sorting note:** [HARP-AI-001] (3.65) is ranked above [LOGIC-003] (3.30) and [SYNTAX-002] (3.25) because [HARP-AI-001] is the readability expression of the same root defect as [LOGIC-001] — an agent encountering it without AST awareness will misread `auth.ts` and reproduce the ordering error. It scores HIGH rather than CRITICAL because no new independent blast radius exists beyond [LOGIC-001].

Bug Triage Board

1. [LOGIC-001] — `ClerkIdentityNotFoundError` class defined after first use; `ReferenceError` on every auth request

- **Severity:** CRITICAL (5/5)
- **Blast Radius:** CRITICAL — affects every single authenticated request in the application; no user can log in successfully while this bug is live
- **Estimated Fix Time:** 5 minutes
- **Fix Pathway:** Patch — move the `class ClerkIdentityNotFoundError extends Error {}` declaration to the top of `auth.ts`, before the `requireAuth` function body
- **Regression Risk:** Negligible — pure declaration reordering; no logic changes; existing tests cover the auth path
- **Why this matters:** JavaScript `class` declarations are not hoisted. The `catch (err) { if (err instanceof ClerkIdentityNotFoundError) }` check inside `requireAuth` (~line 42) references a class that does not yet exist in the execution context at that point (~line 64). At runtime this throws `ReferenceError: ClerkIdentityNotFoundError is not defined` on the first auth call, collapsing the entire anti-resurrection feature and the entire authenticated surface of the API simultaneously.

2. [LOGIC-002] — Non-atomic Clerk + local delete creates permanent orphan row on DB failure; no recovery path

- **Severity:** CRITICAL (5/5)
- **Blast Radius:** HIGH — any user who triggers a DB transient failure during deletion is permanently locked

out (Clerk identity gone, local row stranded); billing is already cancelled; no self-service recovery exists

- **Estimated Fix Time:** 2–4 hours
- **Fix Pathway:** Refactor — introduce a `deletionPending` boolean column on `usersTable`; set it to `true` in a transaction before touching Clerk; have `requireAuth` reject requests where `deletionPending = true` with 401; wrap the `db.delete` in a try/catch post-Clerk-delete and emit a CRITICAL-severity structured log event with the `userId` and `clerkUserId` for manual reconciliation if it fails
- **Regression Risk:** Medium — requires a schema migration and changes to `requireAuth`; must be tested against the concurrent-request and transient-DB-failure paths
- **Why this matters:** After `clerkClient.users.deleteUser` succeeds but before `db.delete(usersTable)` completes, a transient Postgres error leaves a local user row that is permanently unauthenticated (its Clerk identity no longer exists) and cannot be self-deleted by the user. There is no recovery path in the current code. The user retains a billing-cancelled, identity-less record in the database indefinitely, with no operator alert to trigger manual cleanup.

3. [LOGIC-004] — `ensureLocalUser` can silently return `undefined` on concurrent-insert rollback, crashing downstream middleware

- **Severity:** HIGH (4/5)
- **Blast Radius:** HIGH — `req.localUser` becomes `undefined`; every downstream route handler that dereferences `req.localUser!` will throw an unhandled exception; the failure mode is a 500 with no structured error, which could mask the root cause
- **Estimated Fix Time:** 15 minutes
- **Fix Pathway:** Patch — after the post-conflict re-read in `ensureLocalUser`, add: `if (!reread[0]) throw new Error("ensureLocalUser: user row vanished after conflict – concurrent rollback; userId=" + clerkUserId);`
- **Regression Risk:** Low — adds a guard on a code path that is already incorrect (implicit `undefined` return); no behaviour change on the happy path
- **Why this matters:** The `INSERT ... ON CONFLICT DO NOTHING` path was already present before this diff, but the new strict-mode rewrite relies on the return value being a guaranteed `User`. TypeScript's `Promise<User>` return annotation hides the silent `undefined` that falls through when the winning concurrent insert rolls back. Without the explicit throw, the crash surface is a null-dereference deep inside a request handler rather than a clean, logged error at the auth boundary.

What Architect Review Missed

1. [LOGIC-001] — Temporal ordering of `class ClerkIdentityNotFoundError`. Three rounds of architect review passed on a file where a `class` declaration appears *after* the `catch` clause that uses `instanceof` against it. The architects validated the logic of the anti-resurrection guard correctly, but did not catch that the guard is structurally inoperable at